

# Lotul Lărgit de Informatică, Seniori, Ziua 1

Comisia științifică

June 8, 2026

## Problema 1. Gemnuai

Propusă de: Bogdan-Ioan Popa, Daniel Gheorghe

### Soluție completă

Considerăm următoarea soluție, bazându-ne pe tehnica Divide et Impera. La pasul care analizează subsecvența  $le...ri$ , ne dorim să răspundem la query-urile care au capătul stânga între  $le...mid$  și capătul dreapta între  $mid + 1...ri$ . Query-urile care au capătul stânga egal cu capătul dreapta vor fi tratate separat, verificarea fiind trivială.

Considerăm mai întâi subsecvența  $le...mid$ . Pentru fiecare poziție  $le \leq i \leq mid$ , calculăm  $cnt_i$ , pe care îl definim ca fiind cel mai din stânga indice  $j$  posibil ( $mid < j$ ) astfel încât toate intervalele  $C_k, D_k$  din subsecvența  $i...mid$  sunt satisfăcute de către intervalele  $A_k, B_k$  din subsecvența  $i...j$ . Definind asemănător  $cnt_i$  pentru  $mid + 1...ri$ , un query  $st...dr$  va avea răspunsul 1 dacă  $cnt_{st} \leq dr$  și  $st \leq cnt_{dr}$ .

### Calcularea $cnt_i$

Tratăm cazul în care calculăm  $cnt_i$  pentru subsecvența  $le...mid$ , cazul  $mid + 1...ri$  fiind asemănător. Pentru o valoare  $v$  fixată, ne interesează doar trei evenimente, pe care le notăm  $e_1, e_2, e_3$ :

- Prima apariție de la stânga la dreapta a lui  $v$  în  $mid + 1...ri$  într-un interval  $A_k, B_k$ . Prin definiție, dacă nu există o astfel de apariție, o considerăm  $ri + 1$ ,
- Prima apariție de la dreapta la stânga a lui  $v$  în  $le...mid$  într-un interval  $A_k, B_k$ . Prin definiție, dacă nu există o astfel de apariție, o considerăm  $le - 1$ ,
- Prima apariție de la dreapta la stânga a lui  $v$  în  $le...mid$  într-un interval  $C_k, D_k$ . Prin definiție, dacă nu există o astfel de apariție, o considerăm  $le - 1$ .

### Parcurgerea intervalelor

$e_1, e_2, e_3$  pot fi calculate în mod asemănător, pentru că pentru toate trebuie să rezolvăm următoarea subproblemă: Se dau  $T$  intervale  $I_1, I_2, \dots, I_t$ . Pentru fiecare valoare aflată în cel puțin un

interval, care este primul indice în care este acoperită? Pentru a o rezolva, ne vom reține două informații pe care le vom adapta pe parcurs:

- $next_i$  = care este următoarea valoare neacoperită la dreapta lui  $i$ ?
- $acop_i$  = primul indice care a acoperit valoarea  $i$ , 0 altfel

Pentru a actualiza un interval  $[I_{start}, I_{final}]$ , plecăm din  $I_{start}$  și mergem din  $next_i$  în  $next_i$  (marcăm  $acop_i$  la fiecare pas) până ajungem la primul indice mai mare decât  $I_{final}$ . Vom aplica de asemenea path compression. Complexitatea totală va fi  $O(val * \alpha(val))$ , unde  $val$  = lungimea reuniunii intervalelor. Pentru a ajunge de la complexitate  $O(N)$  per pas de divide la complexitate de  $O(ri - le)$ , vom normaliza intervalele la fiecare pas analizat.

## Normalizare

Pentru a avea o complexitate de  $O(N \log N)$  pentru această etapă, vom folosi normalizarea de la nivelul anterior pentru a normaliza pasul actual și facem asemănător cu counting sort. De asemenea, în normalizare vom trata un interval  $[I_{start}, I_{final}]$  ca un interval deschis  $[I_{start}, I_{final} + 1)$ . Asta ne rezolvă un caz de tipul 1...4, 6...10 și în care verificăm dacă 1...10 e acoperit. Dacă am normaliza fără să rezervăm încă o poziție la finalul intervalului, intervalul 1...10 ar fi considerat acoperit. Rezervarea unei poziții în plus rezolvă această problemă.

## Soluția în $O(N \log N + Q)$ offline

Observăm că singurul caz în care vom fi restricționați să avem  $e_1 \leq cnt_i$  este pe intervalul  $e_2 + 1 \dots e_3$ . Dacă  $e_3 \leq e_2$ , nu avem nicio restricție. Astfel, vom avea o serie de intervale care ne influențează pozițiile  $le \dots mid$ . Pentru fiecare poziție, va trebui să răspundem care este intervalul cu cel mai din dreapta  $e_1$ . Această problemă poate fi rezolvată cu o structură de update pe range și query de maxim pe poziție în  $O(\log(N))$  per operație, rezultând o soluție în  $O(N \log^2 N + Q)$ . Pentru a o rezolva în  $O(1)$  amortizat pe poziție, vom parcurge intervalele descrescător după  $e_1$  și vom dori din nou să aflăm primul interval care acoperă o poziție (vezi problema Curcubeu de pe Infoarena). Astfel rezultă o complexitate totală de  $O(N \log N + Q)$ .

## Soluția în $O(N \log N + Q)$ online

Pentru a răspunde online la query-uri, putem reține structura divide-ului în memorie, pe nivele. Complexitatea timp rămâne asemănătoare, iar complexitatea de memorie devine  $N \log N$ . Pentru a trece de la  $O(Q \log Q)$  la  $O(Q)$ , trebuie să aflăm în  $O(1)$  nivelul la care se află un query. Vom considera  $N$  = cea mai mică putere de 2 mai mare sau egală ca  $N$ . Astfel, împărțirea în divide va fi pe puteri de 2. Nivelul la care vom afla răspunsul unui query este most significant bit( $le \text{ xor } ri$ ). Complexitatea totală ajunge acum  $O(N \log N + Q)$ .

## Problema 2. Gepetto

Propusă de: Cătălin Frâncu

### Subtask-ul 1 ( $N \leq 10$ )

Pentru acest subtask, este suficient să se genereze cu backtracking toate permutările *bună* de ordin  $N$  și să se contorizeze numărul celor care sunt mai mici sau egale din punct de vedere lexicografic cu permutarea dată în enunț.

Complexitate:  $O(N! \cdot N)$

### Subtask-ul 2 ( $N \leq 200$ )

O ordine *bună* este, de fapt, o sortare topologică pe graful orientat obținut prin orientarea tuturor muchiilor din arbore de la părinți spre copii.

Notăm cu  $s[u]$  numărul de noduri din subarborele nodului  $u$ .

O formulă foarte utilă este că numărul de sortări topologice într-o pădure de arbori orientați cu  $N$  noduri este egală cu:

$$\frac{N!}{\prod_{i=1}^N s[i]}$$

Această formulă poate fi demonstrată prin inducție matematică.

În continuarea acestei rezolvări, se vor folosi următoarele notații:

- $T$  - arborele orientat dat în problemă
- $T \setminus A$ ,  $A \subseteq \{1, 2, \dots, N\}$  - pădurea de arbori obținută prin eliminarea din arborele  $T$  a nodurilor din mulțimea  $A$ .
- $f(G)$  - numărul de sortări topologice ale pădurii  $G$ .
- $isroot(u, i) = \begin{cases} 1, & \text{dacă } u \text{ este o rădăcină în pădurea } T \setminus \{V_1, V_2, \dots, V_i\} \\ 0, & \text{în caz contrar} \end{cases}$

În alte cuvinte,  $isroot(u, i)$  este răspunsul la întrebarea: "Dacă s-ar elimina nodurile  $V_1, V_2, \dots, V_i$  din arborele dat, ar deveni nodul  $u$  o rădăcină?"

Numărul de ordini bune care sunt strict mai mici lexicografic decât ordinea  $V$  poate fi calculat în felul următor:

$$\sum_{i=2}^N \sum_{u=1}^{V_i-1} isroot(u, i-1) \cdot f(T \setminus \{V_1, V_2, \dots, V_{i-1}, u\})$$

Prin urmare, răspunsul problemei (poziția permutării  $V$  în șirul sortat) este egal cu:

$$1 + \sum_{i=2}^N \sum_{u=1}^{V_i-1} isroot(u, i-1) \cdot f(T \setminus \{V_1, V_2, \dots, V_{i-1}, u\})$$

Deoarece valoarea funcției  $f(G)$  poate fi calculată în  $O(|G|)$ , complexitatea calculării brute a funcției de mai sus este  $O(N^3)$ .

### Subtask-ul 3 ( $N \leq 5000$ )

Din formula numărului de sortări topologice într-o pădure orientată, obținem următoarele consecințe:

$$f(T \setminus \{V_1, V_2, \dots, V_{i-1}\}) = (N - i + 1)! \cdot \frac{\prod_{j=1}^{i-1} s[V_j]}{\prod_{j=1}^N s[j]}$$

$$f(T \setminus \{V_1, V_2, \dots, V_{i-1}, u\}) = (N - i)! \cdot \frac{s[u] \cdot \prod_{j=1}^{i-1} s[V_j]}{\prod_{j=1}^N s[j]}$$

Prin urmare, se obține următoarea egalitate:

$$f(T \setminus \{V_1, V_2, \dots, V_{i-1}, u\}) = s[u] \cdot \frac{f(T \setminus \{V_1, V_2, \dots, V_{i-1}\})}{N - i + 1}$$

Așadar, formula de la subtask-ul anterior poate fi rescrisă în felul următor:

$$1 + \sum_{i=2}^N \frac{f(T \setminus \{V_1, V_2, \dots, V_{i-1}\})}{N - i + 1} \cdot \sum_{u=1}^{V_i-1} isroot(u, i-1) \cdot s[u]$$

Deoarece valoarea funcției  $f(G)$  poate fi calculată în  $O(|G|)$ , complexitatea calculării brute a funcției de mai sus este  $O(N^2)$ .

### Subtask-ul 4 (Există un nod cu $N - 1$ copii)

Particularizând formula de mai sus pe structura arborelui din acest subtask, obținem:

$$1 + \sum_{i=2}^N (N - i)! \cdot \sum_{u=1}^{V_i-1} [1 \text{ dacă } u \notin \{V_1, V_2, \dots, V_{i-1}\}, \text{ altfel } 0]$$

Pentru a calcula eficient formula de mai sus, va fi nevoie de o structură de date  $S$  care să poată efectua eficient următoarele două tipuri de operații:

- 1  $x$ : inserează  $x$  în  $S$ ;
- 2  $M$ : calculează câte numere naturale din intervalul  $[1, M]$  nu fac parte din  $S$ .

Două structuri de date cunoscute care pot efectua ambele tipuri de operații în  $O(\log(N))$  sunt arborii de intervale și arborii indexați binari.

Folosind oricare dintre aceste structuri, se obține complexitatea finală  $O(N \cdot \log(N))$  pentru acest subtask.

**Subtask-urile 5, 6** ( $N \leq 2 \cdot 10^5, N \leq 10^6$ )

Reamintim formula de la subtask-ul 3:

$$1 + \sum_{i=2}^N \frac{f(T \setminus \{V_1, V_2, \dots, V_{i-1}\})}{N - i + 1} \cdot \sum_{u=1}^{V_i-1} isroot(u, i-1) \cdot s[u]$$

Când efectuăm tranziția de la termenul  $i$  la termenul  $i + 1$  din cadrul sumei exterioare, se vor întâmpla următoarele schimbări:

- Primul factor va deveni:

$$\frac{f(T \setminus \{V_1, V_2, \dots, V_{i-1}\})}{N - i + 1} \rightarrow \frac{f(T \setminus \{V_1, V_2, \dots, V_i\})}{N - i} = \frac{f(T \setminus \{V_1, V_2, \dots, V_{i-1}\})}{N - i + 1} \cdot \frac{s[V_i]}{N - i};$$

- $V_i$  nu mai este o rădăcină;
- Toți copiii lui  $V_i$  vor deveni rădăcini.

Similar cu subtask-ul 4, va fi nevoie de o structură de date  $S$  care să poată efectua eficient următoarele tipuri de operații:

- 1 x val:  $S[x] := S[x] + val$ ;
- 2 M: calculează valoarea sumei  $\sum_{i=1}^M S[i]$ .

Două structuri de date cunoscute care pot efectua ambele tipuri de operații în  $O(\log(N))$  sunt arborii de intervale și arborii indexați binari.

Așadar, în funcție de implementare, complexitatea finală pentru acest subtask poate fi  $O(N \cdot \log(N))$  sau  $O(N \cdot \log(N) + N \cdot \log(\text{mod}))$ .

## Problema 3. Printesa

*Propusă de: Andrei Feodorov*

### Observatie

Observatia cheie a problemei este ca daca poligonul poate scapa din inchisoare, acesta poate scapa folosind o singura translatie. Hai sa eliminam o latura si sa consideram o serie de miscari (translatii sau rotatii) care elibereaza poligonul.

Prima miscare nu poate sa fie o rotatie, deoarece fiecare latura ar bloca poligonul. In particular, daca consideram o latura  $(p_1, p_2)$ , orice rotatie l-ar forta ori pe  $p_1$  ori pe  $p_2$  sa iasa din poligon.

Asadar, prima miscare este o translatie, fie aceasta in directia  $d$ , deci putem sa miscam poligonul in directia  $d$  o distanta  $> 0$ . Vom demonstra ca putem sa translatam poligonul in aceasta directie  $d$  o distanta arbitrara. Presupunem prin absurd ca acest lucru nu se poate intampla, deci exista un punct  $p$  si o distanta  $\alpha$  astfel incat segmentul  $(p, p + \alpha d)$  se intersecteaza cu o anumita latura neeliminata a inchisorii (adica, intuitiv, particula  $p$  s-ar intersecta intr-un perete daca translatam in directia  $d$  o distanta  $\alpha$ ). Hai sa consideram acest punct de intersectie (de cea mai apropiata latura). Pentru ca consideram cel mai apropiat punct de intersectie, toate punctele de la  $d$  pana la punctul de intersectie sunt in poligon initial. Pe de o parte, putem sa ne apropiam arbitrar de aproape de acest punct de intersectie si tot o sa fim in poligon. Pe de alta parte, stim ca putem sa translatam poligonul in directia  $d$  cel putin o anumita distanta, prin urmare, contradictie,

Concluzionand, daca putem sa scoatem poligonul, putem sa il scoatem cu o singura translatie.

### Observatie 2

Daca poligonul poate evada eliminand o latura, atunci latura asta trebuie neaparat sa aiba suma unghiurilor adiacente  $\leq 180$ . Intuitiv, incercam sa translatam un obiect mare printr-o gaura mica. Numit aceste tipuri de laturi candidat.

### Poligon convex

Este clar ca daca nu exista o latura candidat, raspunsul e false (asta se aplica si la poligoane concave). O sa demonstram ca daca avem o latura candidat, chiar putem scapa cu o translatie. In particular, putem considera directia unei laturi adiacente. In figura de mai sus, latura  $(0,1)$  este candidat. Consideram directia determinata de latura adiacenta arbitrar aleasa  $(5,0)$ . Consideram cele 2 supporting hyperplanes paralele cu directia (desenate in rosu). Deoarece poligonul este convex si latura aleasa este candidata, hiperplanurile vor intersecta poligonul exact in capetele laturii alese. Asadar, orice punct luam din poligonul convex, daca trasam directia aleasa, o sa intersectam poligonul exact in latura candidata. Asta inseamna ca poligonul poate scapa.

Mai intuitiv, nu exista unghiuri "carlig" care sa ne opreasca din a scoate poligonul pur si simplu printr-o latura candidat.

Laturile candidat se pot gasi in  $O(n)$ .

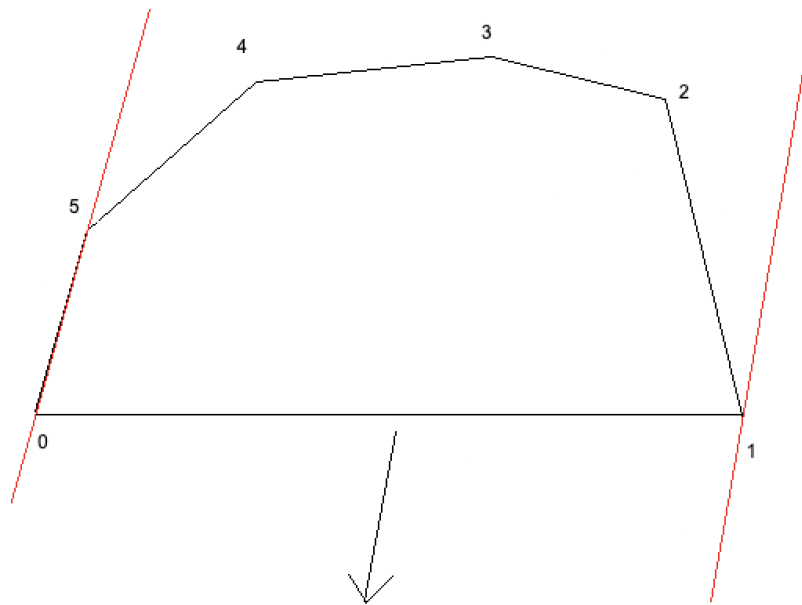


Figure 1: Exemplu convex

## Poligon general $n^2$

Consideram aceeasi idee ca inainte. Sa zicem ca putem sa scoatem poligonul eliminand o latura si translatand intr-o directie  $d$ . Atunci, daca consideram o alta latura, translatand latura asta in directia  $d$ , sigur nu ne vom intersecta cu laturile sale adiacente neeliminate. Intradevar, eliminand o latura, daca avem o directie  $d$  care respecta aceasta proprietate pentru toate laturile neeliminate, atunci  $d$  este o translatie valida.

O latura se poate transla in intr-o directie  $d$  fara sa intersecteze laturile adiacente ale inchisorii doar pe un interval de unghi, dat de vectorii laturilor adiacente. Asta ne duce la solutia de  $n^2$ : Fixam latura pe care o eliminam, calculam intervalul de unghiuri pentru toate celelalte laturi, si calculam intersectia. Daca intersectia este nevida, atunci nu putem iesi distrugand latura asta, altfel putem.

## Poligon general $n \log n$

Similar cu inainte, putem observa ca daca eliminam o muchie, o directie  $d$  este o solutie doar daca  $d$  nu indreapta vreo latura in afara poligonului. In alte cuvinte, putem zice ca o directie  $d$  este valida pentru o muchie, daca directia  $d$  are unghi de 90 sau mai putin cu normala muchiei, si o directie este buna daca este valida pentru toate muchiile neeliminate (vezi figura de mai jos). Asta ne duce la urmatoarea solutie (ineficienta): fixam o muchie, generam normalele celorlalte muchii, si vedem daca exista vreo directie care e in interval de 90 cu toti vectorii. Asta

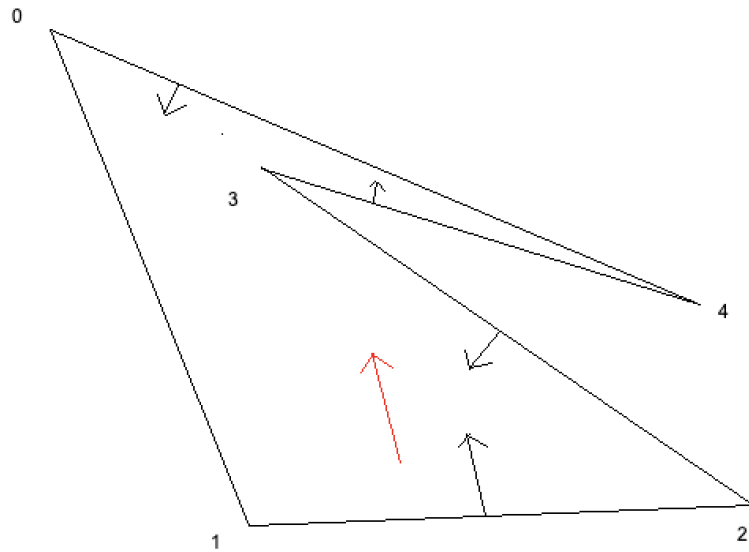


Figure 2: Eliminand muchia (0,1), directia rosie este valida pentru latura (1,2) si (3,4), dar nu pentru (2,3) si (4,0)

e echivalentul cu a vedea daca toti vectorii sunt intr-un interval de 180. Solutia asta e  $n^2$ , dar daca calculam toate normalele, si le sortam dupa tangenta, putem vedea ca putem testa daca o muchie este o solutie de eliminat in  $O(1)$ : putem doar sa ne uitam la vecinul din stanga si cel din dreapta, iar astea reprezinta cei mai departati vectori. Daca acestia au unghi de 180 sau mai putin (masuram unghiul unde nu se afla vectorul eliminat), atunci putem elimina muchia si sa scapam, altfel trebuie sa incercam alta muchie. In total, avem  $n \log n$  de la sortare.

Inca o observatie care ajuta la implementare este ca de fapt nu trebuie sa calculam normalele, ci putem pur si simplu sa folosim vectorii determinati de laturile directionate. Asta este echivalentul normalelor, dar rotite 90, lucru care nu schimba deloc solutia noastra.

## Poligon general $O(n)$

Doar optimizam varianta de mai sus. Se observa ca daca exista un astfel de vector care poate fi eliminat, atunci el este singurul vector dintr-un interval de cel putin 180. Astfel, daca partitionam vectorii in doua multimi, vectori care arata spre stanga si vectori care arata spre dreapta (intuitiv, astea sunt doua intervale diferite de 180 de grade), atunci vectorul care trebuie eliminat(daca exista), este unul dintre cei doi vectori extremi(cel cu tangenta maxima/minima) dintr-o jumatate (vezi figura de mai jos). Asadar, trebuie verificati doar 4 vectori, care pot fi gasiti printr-o simpla parcurgere, deci avem in total  $O(n)$  operatii.

Pentru a eficientiza mai mult si a avea mai putin parcurgeri, putem vedea ca de fapt nu trebuie sa reparametrim iar tot sirul de vectori, ci trebuie doar sa comparam vecinii din stanga si din



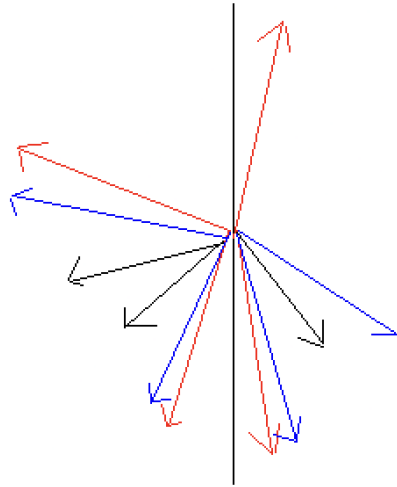


Figure 3: Vectorii in jurul originii, partitionati in 2 multimi. Vectorii rosii sunt cei extremi, vectorii albastri sunt "urmatorii extremi", iar cei negri sunt cei irelevanti

dreapta(ca la solutia cu sortare). Deci, pentru un vector extrem, ne intereseaza extremul din cealalta jumătate si "urmatorul extrem" din jumătatea curenta (vectorii albastri in figure de mai jos). Restul vectorilor sunt irelevanti. Asta duce la o singura parcurgere efectiva a sirului de vectori.

In total, tot avem  $O(n)$  operatii.

## Echipa

Problemele pentru acest baraj au fost pregătite de:

- Adrian Panaete, Colegiul Național "A.T. Laurian", Botoșani
- Nicoli Marius, Colegiul Național "Frații Buzești" Craiova
- Frâncu Cătălin, Nerdvana
- Tamio-Vesa Nakajima, Universitatea Marburg, Hesse
- Andrei-Costin Constantinescu, ETH, Zürich, Elveția
- Alexandru Lorintz, Universitatea "Babeș-Bolyai", Cluj-Napoca
- Toma Ariciu, Universitatea Politehnică București
- Andrei-Cristian Ivan, Universitatea Politehnică București
- Dan-Constantin Spătărel, București
- Vlad-Mihai Bogdan, Universitatea București
- Eugenie-Daniel Posdărăscu, Youni, București
- Radu-Teodor Prosie, Universitatea București
- Liviu Armand Gheorghe, Universitatea București
- Tiberiu Cozma, Universitatea Alexandru Ioan Cuza, Iași
- Ion Andrei Robert, Ecole Polytechnique, Paris
- Cismaru Mihaela, Google Paris
- Perju Verzotti Luca, Ecole Polytechnique, Paris
- Gheorghies Alexandru, Universitatea "Alexandru Ioan Cuza" Iași
- Boacă Andrei, Universitatea "Alexandru Ioan Cuza" Iași
- Daniel Gheorghe, Universitatea din București
- Arsenoiu Iulian, Columbia University
- Feodorov Andrei, ETH, Zürich, Elveția
- Botnaru Victor, Universitatea Politehnica Bucuresti, ACS
- Popescu Adrian, Universitatea Politehnica Bucuresti, ACS
- Măgureanu Livia, JetBrains
- Popa Bogdan-Ioan, Just 4 Kids
- Andrei Chertes, Universitatea Politehnică București